? question

1. Write a programme to find if a number is odd or even ?

Clue: even numbers= 0,2,4,6….. Which are divisible by 2(reminder is zero ,when divided by 2)
Odd numbers are =1,3,5… which are not divisible by 2(reminder is 1 when divided by 2)

Loops in python

Looping means repeating something over and over until a particular condition is satisfied. A for loop in Python is a control flow statement that is used to repeatedly execute a group of statements as long as the condition is satisfied. Such a type of statement is also known as an iterative statement

1. for loop
2. while loop

1. For loop

The for loop in Python is used to iterate over a sequence, which could be a list, tuple, array, or string. The program operates as follows: We have assigned a variable, x, which is going to be a placeholder for every item in our iterable object.

Syntax:

```
for condition(item in a sequence):
    body
```

Example:

```
x=[1,2,3,4]
for i in x:
    print(2*i)
```

```
2
4
6
8
```

2. **range** function

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

Syntax:

range(start, stop+1,difference):

Example:

```
for i in range(11):
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
10
```

```
for i in range(1,11):
    print(i)
```

```
1
2
3
4
5
6
7
8
9
10
```

```
for i in range(1,11,2):
    print(i)
```

```
1
3
5
7
9
```

? write a programme to find the sum of numbers from 1 to 10?

Clue: 1+2+3+4+5+6+7+8+9+10=55

```
sum=0
for i in range(1,11):
    sum=sum+i
print(sum)
```

```
55
```

? write a programme to find the sum of all even numbers from 1 to 10 ?
Clue: 2+4+6+8+10=30

```
sum=0
for i in range(1,11):
    if i%2==0:
        sum=sum+i
print(sum)
```

```
30
```

For Loop in Python

A `for` loop is used to iterate over a sequence (such as a list, tuple, dictionary, set, or string) and execute a block of code for each item in the sequence.

1. `for` Loop Syntax:

```
for variable in sequence:
    # Code block to execute
```

Example:

```
numbers = [1, 2, 3, 4]
for num in numbers:
    print(num)
```

Output:

Copy code

```
1
2
3
4
```

---

2. `range()` Function

The `range()` function generates a sequence of numbers, which is often used with a `for` loop.

- **Syntax:**

python
Copy code

```
range(start, stop, step)
```

- **`start`: Optional. The starting number (default is 0).**
- **`stop`: Required. The ending number (exclusive).**
- **`step`: Optional. The difference between each number in the sequence (default is 1).**

Example:

python
Copy code

```
for i in range(1, 5):
    print(i)
```

Output:

Copy code

```
1
2
3
4
```

---

3. `for` Loop with `else`

The `else` block in a `for` loop executes after the loop completes all its iterations. However, it does not run if the loop is terminated by a `break` statement.

Example:

python
Copy code

```
for i in range(3):
    print(i)
else:
    print("Loop completed")
```

Output:

Copy code

```
0
1
2
Loop completed
```

---

4. `break` Statement

The `break` statement is used to exit the loop prematurely, even if the loop condition is still true.

Example:

python
Copy code

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

Output:

Copy code

```
0
1
2
```

---

5. `continue` Statement

The `continue` statement skips the current iteration of the loop and moves on to the next iteration.

Example:

python
Copy code
```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Output:

Copy code
```
0
1
3
4
```

---

6. Nested `for` Loop

A nested `for` loop is when a `for` loop runs inside another `for` loop.

Example:

python
Copy code
```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j)
```

Output:

Copy code
```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

---

7. `enumerate()` Function

The `enumerate()` function adds a counter to the iteration, returning both the index and the value from the sequence.

Example:

python
Copy code
```
colors = ['red', 'blue', 'green']
for index, color in enumerate(colors):
    print(index, color)
```

Output:

Copy code
```
0 red
1 blue
2 green
```

---

8. List Comprehension

List comprehension provides a concise way to create lists. It consists of brackets containing an expression followed by a `for` clause.

- **Syntax:**

python
Copy code
```
[expression for item in iterable if condition]
```

Example 1: Creating a list of squares.

python
Copy code
```
squares = [x**2 for x in range(5)]
print(squares)
```

Output:

csharp
Copy code
```
[0, 1, 4, 9, 16]
```

Example 2: List comprehension with a condition.

python
Copy code

```
even_numbers = [x for x in range(10) if x % 2 == 0]
print(even_numbers)
```

Output:

csharp
Copy code

```
[0, 2, 4, 6, 8]
```

Summary:

- `for` Loop: Used to iterate over sequences.
- `range()` Function: Generates sequences of numbers for iteration.
- `else` with `for`: Executes when the loop completes normally.
- `break`: Exits the loop.
- `continue`: Skips the current iteration.
- Nested `for` Loops: Loops inside loops.
- `enumerate()`: Adds a counter during iteration.
-